In this chapter, you'll see:

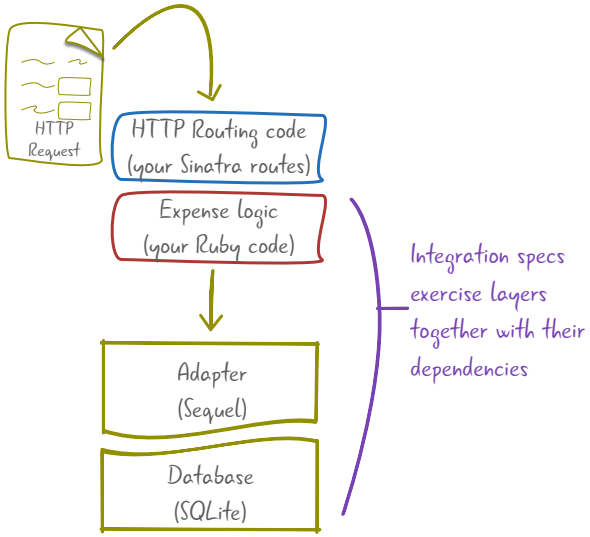
- How to set up a database for testing, without damaging your real data
- Techniques for organizing shared and global setup code
- How to use metadata to control how RSpec runs certain specs
- How to diagnose an ordering dependency between your specs

CHAPTER 6

Getting Real: Integration Specs

By now, you've got a solid HTTP routing layer designed with the help of unit specs. To write these unit specs, you isolated the code under test. Your specs assumed that the underlying dependencies would eventually be implemented, and provided *test doubles*—fake versions for the test.

Now, it's time to write those dependencies for real. In this chapter, you're going to implement the `Ledger` class as the bottom layer of the app. You'll write code to store expense records into a database. You'll create powerful *integration specs* to make sure the data's really getting stored:



By the end of the chapter, not only will your integration specs work, but so will the end-to-end acceptance specs you began this project with.